Verifiable Peephole Optimization for the Execution Layer of Blockchain Information Systems: dMIR Carry-Chain Operator Extension and SMT Equivalence Checking

Anonymous Author(s)

Anonymous Institution
anonymous@example.com

Abstract. Blockchain JIT peephole rewrites must strictly preserve transaction semantics bit-for-bit, as any divergence in consensus VMs will induce consensus forks. In LLVM IR, extracting carry and overflow from aggregate intrinsic returns fragments carry-dependent rewrites across SSA nodes, blocking single Alive2 queries and compelling engineers to adopt suboptimal trade-offs (sacrificing coverage, safety, or efficiency). We elevate carry and overflow to first-class bit-vector operators in dMIR (DTVM's middle IR), with Z3-aligned operators enabling single Z3 equivalence queries for rewrites and a two-tier system. An offline Z3-verified synthesis pipeline covers the deployed rule scope via canonical-form matching, rediscovering all 70 production dMIR rules; the production set augments these with 13 differentially tested x86 rules. The system achieves a 7.24% geometric-mean speedup on 27 evmone-bench workloads, with 5884 differential Cancun tests reporting byte-identical state and gas across the suite (correctness bounded by suite coverage). It resolves verification bottlenecks for consensus VM peephole rewrites; more broadly, domain-specific IR with first-class bit-vector operators provides a reusable path for verifiable optimization in consensus-critical engines.

Keywords: Blockchain information systems · EVM JIT · dMIR · Peephole optimization · SMT equivalence checking · Carry chain · Translation validation

1 Introduction

The execution layer of a blockchain information system is a replicated consensus state machine: every node on the consensus path must produce bit-identical results for the same transaction. The EVM is the most widely deployed smart-contract execution environment, and its JIT compiler is progressively replacing the interpreter in mainstream clients, becoming a core component on the consensus-critical path. Peephole rewrites in the JIT act directly at machine-code generation, so their correctness bears directly on the determinism of on-chain execution; a wrong equivalence judgment, deployed on only some clients, may trigger a chain fork.

Existing formal verification of EVM peephole rewrites mostly targets the bytecode block-level abstraction. The machine-level patterns emitted by a JIT during code generation—especially the multi-word carry chains that arise when the 256-bit word length is lowered onto 64-bit hardware—sit at a different abstraction layer, and an end-to-end automatically verifiable rule set with a concrete deployment path is still missing. The root cause is how IRs model bit-level side-products. In LLVM IR, basic arithmetic is defined only on N-bit integers; the carry bit must be retrieved through an intrinsic that returns a multi-return tuple of {result, carry}. Rules depending on this side-product can be expressed only through indirect structures in Alive [13] and similar tools; the bottleneck is the IR's expressiveness, not the SMT solver [14]. For a consensus-critical JIT, an unverifiable rule class leaves the engineer with three choices—drop it, ship it without verification, or rely on a hand-written paper proof—each sacrificing performance, consensus safety, or scalability.

Our approach: extend the IR with first-class bit-vector operators so that SMT-verifiable peephole rules cover a larger surface, without modifying the solver or relaxing the differential-test threshold. In dMIR, the bit-level patterns that previously required indirect multi-return encoding become first-class operators, so the corresponding rules fall within the decidable fragment of Z3 bit-vector equivalence.

Contributions. The contributions of this paper are as follows:

1. **An automatically verifiable peephole system through IR-side extension.** Bit-level side-products that a general-purpose IR can only carry through a multi-return tuple become first-class bit-vector operators of dMIR. Without modifying the solver and without loosening the differential-test threshold, the IR extension moves a rule class that previously sat outside the Alive2 automatic equivalence-verification framework into the decidable fragment of the Z3 bit-vector equivalence check [14]. A two-tier rewrite system is built on **70** dMIR production rules (4 of which rely on the first-class modeling and cannot be expressed as a single peephole rule on general IR) together with **13** rules at the x86 code-generation layer (covered by differential tests; no separate SMT check).
2. **Capacity, admission rate, and production-rule coverage of the synthesis pipeline.** A capacity run evaluates **853** candidates under Z3 admission (**98.7%** algebraic, **21.8%** carry); a separate coverage run admits **1966** rules whose canonical LHSs cover all **70/70** hand-designed production rules (enumerator extended from an initial 0/70 baseline; see §4.1).
3. **End-to-end performance and correctness.** Across the **27** EVM benchmarks of evmone-bench, the geometric-mean speedup is **+7.24%** (95% confidence interval [+2.59%, +12.11%], lower bound significantly greater than zero), with a peak of **+25.79%**; **5884** differential tests agree byte-for-byte on state and gas between the rules-enabled and rules-disabled sides (DTVM self-A/B, same engine).

Relative to the EVM bytecode block-level formal rule set of Albert et al. [2] at CAV 2023, the rule set in this paper operates at the JIT intermediate representation and machine-code layers that the bytecode abstraction cannot reach (quantitative comparison in §5).

2 Background and Motivation

2.1 Hardware Cost and Carry-Chain Shape of U256 Arithmetic

The native word length of the EVM is 256 bits, whereas general-purpose registers on current commodity hardware are 64 bits wide. An EVM-level U256 addition expands, after JIT code generation, into four machine-level additions: one ordinary 64-bit `add` followed by three carry-propagating `adc` instructions that form a *serial carry chain*. Subtraction is analogous, using one `sub` and three `sbb` instructions. The listing below shows the 4-limb code on both sides for a typical U256 addition:

dMIR (4-limb addition)	**x86-64 output**
`r0 = add(a0, b0)`	`add rax, rcx`
`r1 = ADC(a1, b1, cout0)`	`adc rbx, rdx`
`r2 = ADC(a2, b2, cout1)`	`adc rsi, rdi`
`r3 = ADC(a3, b3, cout2)`	`adc r8, r9`

Among all instructions emitted by the JIT, instructions directly tied to U256 multi-word arithmetic account for about **2.15%** (weighted mean across the U256-heavy subset of the 27 benchmarks in §4—the leftmost cluster "U256-heavy (1–11)" in Figure 1). The share is small, but the carry chain has a special property: every `adc`/`sbb` on the chain depends on the carry output of the previous instruction, forming a strict *serial data dependency* that instruction-level parallelism cannot hide. U256 instructions are therefore few but contribute disproportionately to execution time. Section 4.2 reports per-family hit rates: on U256-heavy benchmarks the U256 rule hit rate ranges from 25% to 99.9% with a very uneven distribution; on non-U256-heavy benchmarks (sha1 family, `jump_around`) it is well below that range, and gains come from x86-layer mechanical cleanup.

2.2 Expressiveness Limit of General-Purpose IR on the Carry Chain

In LLVM IR, basic arithmetic (`add`, `sub`, `mul`) is defined only on N-bit integers `iN`; to obtain the carry bit, the programmer must call `@llvm.uadd.with.overflow.iN`, which returns a `{iN, i1}` multi-return tuple. The carry is *not a native type* but hidden in the second field of a struct. Following the Alive/Alive2 [13,12] peephole verification framework, Table 1 lists four carry-chain rules that Alive2 cannot directly verify (type mismatch on the multi-return tuple, or timeout on the lifted encoding); the limitation stems from IR carry modeling, independent of solver capacity.

Table 1. Four typical carry-chain rules that Alive2 (based on LLVM IR) cannot directly verify for equivalence because of the multi-return tuple encoding. The first three are single `ADC`/`SBB` rules where the right-hand side drops the carry output; the fourth is a 4-limb carry-chain merge rule.

Rule	Rewrite
`adc_zero_carry`	`ADC`$(x, y, 0) \rightarrow$ `add`(x, y)
`sbb_zero_borrow`	`SBB`$(x, y, 0) \rightarrow$ `sub`(x, y)
`sbb_self_zero`	`SBB`$(x, x, 0) \rightarrow 0$
4-limb ADC chain	$4 \times$ `uadd.w.overflow` $\rightarrow$ `add.i256`

The Alive2 command lines, failure messages, and versions (Alive2 mainline v21.0, Z3 4.8.12) for all four rules are archived in the artifact (§6); the relation to Hydra [16] (OOPSLA 2024, distinct from the Hydra Framework at USENIX Security 2018) is discussed in §5. The bottleneck is not the SMT solver [14]: the remaining six U256 rules without multi-return carry pass Alive2 in ∼60 ms average. Expressing carry-propagating addition in LLVM IR requires five lines around `uadd.with.overflow` to extract result and carry from a tuple, whereas a single `ADC` suffices in dMIR; native carry-bit support brings all four rules of Table 1 into the scope of automatic equivalence checking under the dMIR bit-vector model.

3 SMT-Verifiable Peephole Rewrite Synthesis on dMIR

The pipeline runs offline rule synthesis once and emits an SMT-admitted JSON rule pack; at JIT init the two-tier runtime—dMIR for algebraic rewrites, x86 CgIR (the code-generation IR) for carry-chain cleanup—loads it. The dMIR bit-vector model is in §3.1; the synthesis pipeline in §3.2.

3.1 dMIR Bit-Vector Semantics and Carry-Chain Operators

The core data type of dMIR is a parametric-width bit-vector BV_n with BV_1 carrying the carry in/out (c_{in}, c_{out}). The carry-propagating addition has signature `ADC` $: \mathrm{BV}_n \times \mathrm{BV}_n \times \mathrm{BV}_1 \rightarrow \mathrm{BV}_n \times \mathrm{BV}_1$; `SBB` is symmetric. In Z3 [14], carry propagation zero-extends both 256-bit operands and the 1-bit carry to 257 bits, sums once, and slices: `wide = ZeroExt(1,x) + ZeroExt(1,y) + ZeroExt(256,cin)`; result is `Extract(255,0,wide)`, carry out is `Extract(256,256,wide)`. This embeds carry propagation entirely in bit-vector arithmetic with no multi-return tuple or auxiliary predicate. The same semantics covers ordinary operators (`add`, `sub`, `mul`, `and`, `or`, `xor`, shifts); the carry-chain operator is the piece a general-purpose IR lacks.

Trusted base. The SMT check assumes the dMIR bit-vector model agrees with the JIT runtime; the 5884 differential tests of §4.3 show no divergence (random-bytecode fuzzing is future work, §5.3).

3.2 Candidate Enumeration and SMT Admission

Admission check. For a candidate $LHS \to RHS$, we submit to Z3 the proposition $\exists\, \boldsymbol{v}.\ LHS(\boldsymbol{v}) \neq RHS(\boldsymbol{v})$. *Unsat* grants admission under fixed-width modular bit-vector semantics; *sat* or timeout rejects (sat returns a counterexample). Equivalence assumes nothing beyond modular arithmetic—no two's complement, no signed overflow. All 70 production dMIR rules pass; queries are committed with each rule and re-run by CI on every change.

Two enumeration runs. We report two separate enumeration experiments with disjoint goals. The *capacity run* fixes a single grammar—a depth-2 algebraic enumerator plus a small set of hand-written carry templates—and measures the Z3 admission rate; this characterizes solver throughput and the typical false-positive boundary. The *coverage run* extends the same algebraic enumerator with additional left-hand-side operator tops and a constant-pool widening, plus a pattern-template instantiation channel reusing the seed-miner configuration; this measures rediscovery of the 70 hand-designed dMIR production rules; coverage results appear in §4.

The capacity run uses two complementary paths: *(i) depth-2 algebraic enumeration*—all left/right combinations of depth ≤ 2 over the dMIR arithmetic and Boolean operators, covering common algebraic identities; *(ii) carry-template enumeration*—multi-word carry-chain candidates constructed by hand around the carry-chain operators.

Throughput and admission rate. The capacity run admits 765/775 algebraic candidates (98.7%, 10 timeouts) and 17/78 carry templates (21.8%, 61 semantic rejections); the gap shows hand-written carry templates routinely reach the non-equivalence boundary where the SMT check is essential. Full data: Table 3 (§4).

Production rule coverage (methodology). The coverage run asks a different question: does the synthesis search space contain the 70 hand-designed production dMIR rules? We compare canonicalized left-hand sides. The *alpha-normalized* canonical form renames variables to a fixed sequence $x_1, x_2, \ldots$ in traversal order, so that two rules differing only in operand naming map to the same representative; commutative pairs (e.g. $\texttt{or}(x, y)$ and $\texttt{or}(y, x)$) likewise collapse to one representative. A production rule counts as *rediscovered* when at least one synthesized rule has the same canonical left-hand side; the *false-miss rate* is the fraction of production rules with no such match. The match is on canonical left-hand sides only—the synthesized right-hand side is verified equivalent to the left-hand side by Z3 but is not required to be syntactically identical to the production rule's right-hand side. Numerical results appear in §4.

Counterexamples. Two rejected carry templates illustrate why SMT admission is essential on the carry chain. $\texttt{SBB}(x, x, c_f) \to 0$ fails at $x{=}0, c_f{=}1$ (LHS is $\texttt{0xFF}\ldots\texttt{F}$; correct form $\texttt{sub}(0, c_f)$); $\texttt{SBB}(x, 0, c_f) \to \texttt{sub}(0, c_f)$ fails at $x{=}1, c_f{=}0$ (correct form $\texttt{sub}(x, c_f)$). Both reflect modular-carry traps: ignored carry input and confused operand polarity. The artifact ships ~2400 lines of Python

Table 2. Classification of the two-tier peephole rewrite rules. The dMIR rules are checked for equivalence by Z3 on the bit-vector semantic model; the x86 CgIR rules are covered by the matched test suites.

Layer (verification)	Category	Count
dMIR (Z3 check)	Boolean normalization	42
	Algebraic identity	15
	Identity elimination	10
	Constant folding	2
	Dead-code elimination	1
	Subtotal	**70**
x86 CgIR (test coverage)	Redundant compare/test cleanup	8
	Branch-fallthrough cleanup	2
	No-op elimination	2
	test-setcc merge	1
	Subtotal	**13**
Total		**83**

(enumerator, seed-miner, canonical comparator, Z3 queries), the rule-set JSON, coverage output, and unit tests for canonicalization/dedup.

Two-tier rule set. The dMIR layer handles machine-independent algebraic simplification (all 70 rules Z3-verified); the x86 CgIR layer handles hardware-specific cleanup on registers, flags, and branches (13 mechanical syntactic rewrites with **no separate SMT script**, covered by the matched 5884-test suites of §4.3). A formal x86 model with SMT check is future work (§5.3).

Table 2 classifies the rules. On the dMIR layer, Boolean normalization dominates (42 rules, 60%, broadly covered by algebraic enumeration); algebraic identities (15) include add-zero and mul-one; identity-elimination (10) covers forms like $\texttt{sub}(x, x) \to 0$ and $\texttt{and}(x, x) \to x$. The x86 layer is dominated by redundant compare/test cleanup (8 rules), with 2 each for branch-fallthrough and no-op elimination. The two layers are complementary: dMIR rules dominate U256-heavy contracts (e.g. `weierstrudel`), x86 cleanup dominates sha1; per-family hit rates are reported in §4.

Asymmetric verification workflow. The 70 dMIR production rules are designed manually from runtime performance profiles and submitted one by one to the Z3 equivalence check (§3.2). The synthesis pipeline is an independent measurement channel: it does not drive production rule maintenance, although coverage results in §4 show its search space contains the production set; production rule selection remains driven by runtime profiles rather than enumeration order, so the two channels are kept separate by selection criterion rather than by expressive scope. The x86-layer rules are covered by code review and the matched test suites. The limitations from this asymmetry are listed explicitly in §5.3.

Table 3. Output of the synthesis pipeline: candidate counts, Z3 passes, rejections, and admission rate. The two enumeration paths are described in detail in §3.2.

Candidate source	Input	Pass	Reject	Admission rate
Algebraic enumeration	775	765	10*	98.7%
Carry templates	78	17	61	21.8%

* 10 candidates returned neither sat nor unsat within the time limit and are conservatively rejected.

4 Evaluation

4.1 Rule Synthesis Output Against the General-Purpose IR

Table 3 reports the synthesis pipeline output of §3.2; Alive2's verification on the same rule sample is summarized in the prose below.

Z3 averages ∼60 ms per 256-bit bit-vector query and discharges all 70 dMIR rules in seconds, so 256-bit width is not a solver bottleneck. On a 10-rule U256 sample from the capacity run, Alive2 clears the 6 without multi-return carry but reports type mismatch or times out on the 4 with `adc`/`sbb` structure (independent of Table 1 in §2.2); Z3 discharges all 10 under the dMIR semantics.

Production rule coverage. Using the canonical-LHS methodology of §3.2, the coverage run produces **1966** Z3-verified rewrite rules: **1856** from the extended algebraic enumerator and **110** from pattern-template instantiation. The pattern-template path starts from 2964 candidates, which Z3 verification and deduplication against the enumerator output and the existing rule file reduce to the final 110. On the 70 hand-designed production rules, the run matches **69/69** unique canonical patterns—one commutative-twin pair, `or`(x, y) versus `or`(y, x), shares a single canonical representative that both production names match—equivalently **70/70** production rule names; the false-miss rate against the production set is therefore **0/70**.

Scope of the coverage claim. Two boundaries should be stated clearly. First, the result measures coverage of the *existing* production rule set; the extended enumerator's left-hand-side operator tops and constant pool were broadened in light of an earlier 0/70 baseline that exposed mechanical gaps (missing tops such as `not`/`select`/`shl`/`ushr`/`sshr` as left-hand-side roots, missing small constants, and structural-equality dedup of forms such as `and`(x, x)). Reaching 70/70 therefore demonstrates that the production rules lie inside an enumerable and Z3-discharged search space; it does not demonstrate blind recall of an unseen target. Second, rediscovery is defined on canonical left-hand sides; the synthesized right-hand side is verified equivalent to its own left-hand side by Z3 but is not required to be syntactically identical to the production rule's right-hand side. Together with the admission rate above, this result shows the search space is broad enough to contain every hand-designed rule, while 1966 of its members pass Z3 verification.

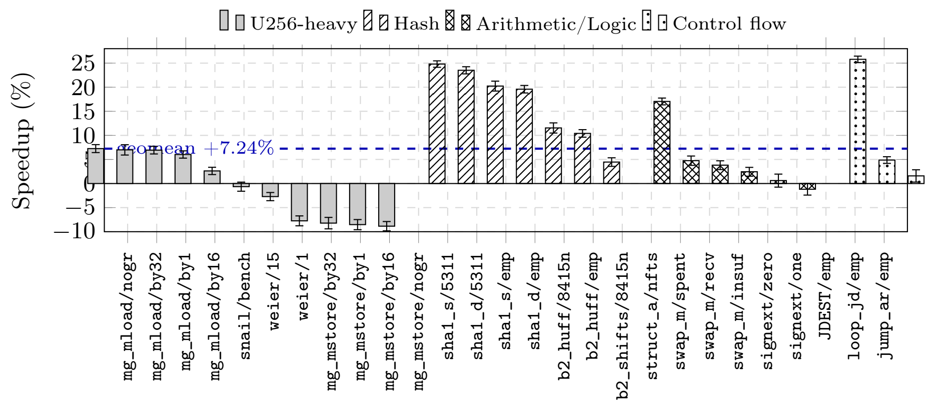

Fig. 1. Speedup on 27 benchmarks from enabling the peephole rewrite rule set over the disabled baseline (error bars are 95% confidence intervals). The horizontal line at zero marks the no-change baseline; the dashed line at **+7.24%** marks the geometric-mean speedup across all 27 benchmarks. From left to right, the benchmarks are grouped into four clusters by workload type: U256-heavy (1–11), hash (12–18), arithmetic/logic (19–24), and control flow (25–27).

4.2 End-to-End Performance

Setup. Baseline is upstream main (2 hard-coded x86 rules); the enabled side adds the 83 new rules (70 dMIR + 13 x86). Both run under evmone-bench, 20 repetitions, bootstrap geomean 95% CIs (B=10 000); the process is pinned with `taskset -c 2-3` at `nice -5`, turbo disabled, load <0.5; CV $\geq 3\%$ runs are rerun at 30 reps.

Across the 27 benchmarks (Figure 1), the enabled rule set delivers a geometric-mean speedup of **+7.24%** (95% CI [+2.59%, +12.11%]), peaking at **+25.79%** on `JUMPDEST_n0/empty`; `sha1` family gains +19.59%–+24.81% and `blake2b_huff` +10.41%–+11.54%. U256 rule hit rates split sharply by workload (∼99.9% on `mg_mload`/`mg_mstore`, 33.8%/25.3% on `weierstrudel`/`snailtracer`, 7.5% on sha1), so sha1/`blake2b` gains come from x86 branch/compare-merge rules instead. Hit rate and speedup are not proportional—`mg_mload` matches almost every instruction yet per-rewrite saving is small—confirming the two layers are complementary: multi-word rules concentrate gains on U256-heavy contracts, x86 cleanup gives broader ordinary gains.

Regressed surface. Five benchmarks regress beyond −2%: the 4 `memory_grow_mstore` variants by 7.74%–8.85% and `weierstrudel/1` by 2.71% (CV < 2%). The 4 `mstore` variants cluster on the same write path (§5.3, item 5); the matched `mload` siblings accelerate by +6.03%–+7.25%. Regressions appear only in a ∼0.7 ns/op microbenchmark, not on the macro sha1/`blake2b` gains.

Table 4. Results on four differential test categories for the enabled side (83 new rules) and the baseline side (upstream main): passes/total. Identical numbers in both columns indicate that the enabled rule set introduces no correctness regression.

Test dimension	Enabled	Baseline	Suite source
Unit (JIT)	223/223	223/223	EVM spec subset
Unit (interpreter)	215/215	215/215	Run list
statetest (JIT)	2723/2723	2723/2723	Cancun 101 suites
statetest (interpreter)	2723/2723	2723/2723	Cancun 101 suites
Total	**5884/5884**	**5884/5884**	—

Compilation overhead. Across 776 JIT unit-test modules, median per-module compile time is 0.45 ms (p95 0.87 ms) covering the full loading–analysis–codegen pipeline; this is an upper bound on the whole pipeline rather than the incremental rewrite-pass cost (per-pass attribution is future work, §5.3, item 4).

4.3 Correctness and Rule Coverage

Both sides run four differential categories (Table 4) and agree item-by-item on state and gas, for **5884/5884** total. The baseline pass validates the suite; the enabled pass shows the 83 new rules introduce no regression. `evmone-statetest` runs independently under multipass JIT and interpreter on Cancun (`-k fork_Cancun` filters Prague), and the two modes agree on the same matrix—direct evidence of EVM consensus consistency.

CI re-verifies each dMIR rule's stored Z3 query on every change; all 70 rules re-verify in ∼0.5 s offline (distinct from the runtime compilation latency of §4.2). Comparison with the CAV 2023 rule set of Albert et al. [2] appears in §5.

5 Related Work and Discussion

5.1 Verifiable Peephole Optimization Frameworks

We position our work along three axes: abstraction layer, verification method, and carry-chain coverage. The synthesize-then-verify paradigm originates with Denali [10] and Bansal–Aiken [5]; STOKE [20] and Stratified Synthesis [8] pursue stochastic and learning variants, while egg [21] offers a non-SMT route via equality saturation. SMT-based verifiable peephole optimization is exemplified by Alive [13], Alive2 [12], Souper [18], and PEEK [17], with dataflow filtering [15] accelerating the pipeline. We follow this paradigm but shift the IR under verification from LLVM IR to dMIR; Alive itself flagged LLVM's multi-return carry/overflow encoding as an expressiveness limit, which is our starting point.

LeanMLIR [6] formalizes IR-parametric peephole verification in Lean 4—complementary to ours: it targets framework generality rather than an instantiated, end-to-end JIT rule set. Hydra [16] discovers bit-width-independent rules but, like Alive2, builds on LLVM IR and inherits the multi-word-carry limit (§2). Schett and Nagele [19] apply offline enumeration plus SMT to EVM *bytecode*, generating ∼993 rules with comparable equivalence guarantees; we instead operate on the JIT code-generation path with structured dMIR matching and x86 cleanup. Combining the two as independent passes is future work. Recent LLM-driven peephole synthesis on LLVM IR [22] is a generative complement to our deductive enumeration on a domain IR with first-class carry operators.

5.2 Verification Work on the Consensus-Critical Path

Albert et al. [2] give a Coq proof of an AOT rule set on EVM bytecode blocks via offline stack-layer bit-vector identities. Our scope differs: JIT IR and machine-code layers, SMT admission at synthesis time, multi-word carry chains. Their 14 `forves` rules overlap our 83 in 7 standard identities any SMT-verified peephole system contains; the remaining 76 (13 x86-layer, 63 dMIR-layer) cover decomposed carry chains, `ADC`/`SBB` simplifications, and code-generation-tied constant propagation and strength reduction.

Runtime divergence detection—the Hydra Framework [7] (distinct from [16]), multi-client detectors, EVM fuzzing, consensus test networks—catches faults after deployment; we harden statically. Bytecode-side EVM optimization [3,9,1,4,11] operates on contract or bytecode-block boundaries, orthogonal to our JIT IR/code-generation focus.

5.3 Limitations and Future Work

(1) Trusted base and x86 coverage. The SMT check rests on the dMIR bit-vector model, assumed to agree with runtime JIT semantics; the 13 x86 rules are mechanical rewrites whose correctness relies on paired differential runs (§4.3). A formal x86 model and systematic dMIR-vs-runtime testing are future work. **(2) Workload and modeling scope.** Speedups come from 27 evmone-bench benchmarks; only arithmetic and flag semantics are SMT-checked, leaving gas, memory aliasing, and storage side effects unmodeled. **(3) Dead-carry inertness at U256 heads.** The head of a U256 add/sub chain has a runtime third operand rather than a matchable zero, so the rule is inert there by construction; it is retained because it remains effective inside U256 multiplication lowering. **(4) Compilation-overhead measurement.** Median 0.45 ms and p95 0.87 ms per module are single-side; EVMC has no per-stage timing hook, so we report only an upper bound. **(5) `memory_grow_mstore` regression.** The 4 `mstore` variants slow down by 7.74–8.85% (CV <2%); preliminary analysis localizes this to `optimizeCmp` failing to match the `SETCC`→`SUBREG_TO_REG`→`TEST`→`JCC` chain on the memory-expand path; the 4 `mload` siblings accelerate by +6.03–+7.26%; root cause is future work.

6 Conclusion

This paper builds a two-tier peephole rewrite system for the deterministic EVM JIT: the 70 dMIR rules pass the Z3 bit-vector equivalence check, and the 13 x86 rules are covered by paired differential runs. A small intersection with Albert et al. [2] on bytecode-level identities is complementary; the methodology—first-class carry/overflow operators in a domain-specific IR so that automatic equivalence checking covers a larger surface—generalizes to other deterministic execution environments with multi-word arithmetic (Wasm big-integer, zk-VM back-ends). Future work includes a formal x86 model, dataflow-based pruning to scale beyond the 853-candidate capacity run and the 1966-rule coverage run reported here, and an orthogonal-composition study that quantifies the additional speedup from layering bytecode-level rules on top of ours.

Data and Code Availability

Rule synthesis scripts, rule sets (JSON + Z3 queries), evaluation scripts, and experimental data are released through an anonymous mirror; the URL is in the double-blind submission materials and will be de-anonymized upon acceptance. CI re-runs the equivalence check on every rule change.

References

1. Aguiar, M.A., Albert, E., Genaim, S., Gordillo, P., Hernández-Cerezo, A., Kirchner, D., Rubio, A.: Neural-guided superoptimization in Ethereum. Information and Software Technology (2025). https://doi.org/10.1016/j.infsof.2025.107800
2. Albert, E., Genaim, S., Kirchner, D., Martin-Martin, E.: Formally verified EVM block-optimizations. In: Computer Aided Verification (CAV). LNCS, vol. 13966, pp. 176–189 (2023). https://doi.org/10.1007/978-3-031-37709-9_9
3. Albert, E., Gordillo, P., Rubio, A., Schett, M.A.: Synthesis of super-optimized smart contracts using Max-SMT. In: Computer Aided Verification (CAV). LNCS (2020). https://doi.org/10.1007/978-3-030-53288-8_10
4. Avigad, J., Goldberg, L., Levit, D., Seginer, Y., Titelman, A.: A proof-producing compiler for blockchain applications. Journal of Automated Reasoning (2025). https://doi.org/10.1007/s10817-025-09723-y
5. Bansal, S., Aiken, A.: Automatic generation of peephole superoptimizers. In: Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) (2006). https://doi.org/10.1145/1168857.1168906
6. Bhat, S., Keizer, A., Hughes, C., Goens, A., Grosser, T.: Verifying peephole rewriting in SSA compiler IRs. In: 15th International Conference on Interactive Theorem Proving (ITP). LIPIcs, vol. 309, pp. 9:1–9:20 (2024). https://doi.org/10.4230/LIPIcs.ITP.2024.9
7. Breidenbach, L., Daian, P., Tramèr, F., Juels, A.: Hydra: Principled bug bounties for smart contract security. In: 27th USENIX Security Symposium (USENIX Security 18) (2018)

8. Heule, S., Schkufza, E., Sharma, R., Aiken, A.: Stratified synthesis: Automatically learning the x86-64 instruction set. In: Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI) (2016). https://doi.org/10.1145/2908080.2908121
9. Hu, X., Zhao, D., Scholz, B.: Synthesizing efficient super-instruction sets for Ethereum virtual machine. In: Proceedings of the 16th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages (VMIL) (2024). https://doi.org/10.1145/3689490.3690403
10. Joshi, R., Nelson, G., Randall, K.: Denali: A goal-directed superoptimizer. In: Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation (PLDI) (2002). https://doi.org/10.1145/512529.512566
11. Krijnen, J.O., Chakravarty, M.M., Keller, G., Swierstra, W.: Translation certification for smart contracts. Science of Computer Programming (2024). https://doi.org/10.1016/j.scico.2023.103051
12. Lopes, N.P., Lee, J., Hur, C.K., Liu, Z., Regehr, J.: Alive2: Bounded translation validation for LLVM. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI) (2021). https://doi.org/10.1145/3453483.3454030
13. Lopes, N.P., Menendez, D., Nagarakatte, S., Regehr, J.: Provably correct peephole optimizations with alive. In: Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI) (2015). https://doi.org/10.1145/2737924.2737965
14. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (2008). https://doi.org/10.1007/978-3-540-78800-3_24
15. Mukherjee, M., Kant, P., Liu, Z., Regehr, J.: Dataflow-based pruning for speeding up superoptimization. Proc. ACM Program. Lang. **4**(OOPSLA) (2020). https://doi.org/10.1145/3428245
16. Mukherjee, M., Regehr, J.: Hydra: Generalizing peephole optimizations with program synthesis. Proc. ACM Program. Lang. **8**(OOPSLA1) (2024). https://doi.org/10.1145/3649837
17. Mullen, E., Zuniga, D., Tatlock, Z., Grossman, D.: Verified peephole optimizations for CompCert. In: Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI) (2016). https://doi.org/10.1145/2908080.2908109
18. Sasnauskas, R., Chen, Y., Collingbourne, P., Ketema, J., Taneja, J., Regehr, J.: Souper: A synthesizing superoptimizer (2017)
19. Schett, M.A., Nagele, J.: Populating the peephole optimizer of a smart contract compiler. In: 2nd Workshop on Formal Methods for Blockchains (FMBC 2020). OASIcs (2020). https://doi.org/10.4230/OASIcs.FMBC.2020.3
20. Schkufza, E., Sharma, R., Aiken, A.: Stochastic superoptimization. In: Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) (2013). https://doi.org/10.1145/2451116.2451150
21. Willsey, M., Nandi, C., Wang, Y.R., Flatt, O., Tatlock, Z., Panchekha, P.: egg: Fast and extensible equality saturation. Proc. ACM Program. Lang. **5**(POPL) (2021). https://doi.org/10.1145/3434304
22. Xu, Y., Xu, H., Tian, S., Zhou, Y., Sun, M.: LPO: Discovering missed peephole optimizations with large language models. In: Proc. ASPLOS (2026), arXiv preprint arXiv:2508.16125